Cheatsheet — Hashcat

Pentesting · Command reference

1. Hash identification

Command	What it does
`hashid '<hash>'`	Returns the hash type, ordered from most to least likely (may return more than one).
`hashid '<hash>' -m`	Returns the hash type together with the module number used in hashcat.
`hashid hash.txt`	Returns the type of the hash contained in the text file (only one hash per file).
`hashcat --example-hashes`	Shows example hashes from hashcat; useful to compare when unsure of the type.
`hashcat --help | grep '<Tipo_hash>'`	Gets the hash module number when its name is known.

2. Optimization and performance

Command	What it does
`hashcat -b -m <modulo>`	Benchmark: the speed at which the hash will be cracked; helps gauge difficulty before starting.
`hashcat -O`	Increases cracking speed, but limits the password length.
`hashcat -w 3`	Increases speed; recommended only when hashcat is the only running process (default 2; use 1 for background).

3. Attack types

Command	What it does
`hashcat -a 0 -m <modulo> hash.txt <diccionario>`	Dictionary attack: uses the provided list to try to crack the password.
`hashcat -a 1 -m <modulo> hash.txt <dicc1> <dicc2>`	Combination attack: combines two dictionaries in all possible ways.
`hashcat -a 3 -m <modulo> hash.txt "<máscara>"`	Mask attack: the password follows a pattern; all possibilities of the pattern are tried.
`hashcat -a 6 -m <modulo> hash.txt <diccionario> <máscara>`	Hybrid attack: each dictionary word is appended with the mask.
`hashcat -a 7 -m <modulo> hash.txt <máscara> <diccionario>`	Hybrid attack: each dictionary word is prefixed with the mask.

Mask characters

- ?l: lowercase letters
- ?u: uppercase letters
- ?d: digits 0 to 9
- ?h: 0123456789abcdef
- ?H: 0123456789ABCDEF
- ?s: all special characters
- ?a: all possible characters

Example: "DOMAIN?l?l?l?l19?d?d" → passwords like DOMAINabcd1900, where abcd is any lowercase letter and the year ranges from 1900 to 1999.

Custom charsets

Can be defined with the -1 to -4 flags; syntax: -1 <chars>. E.g.: -1 01 with "DOMAIN?l?l?l?l20?1?d" yields years between 2000 and 2019. Up to 4 custom charsets.

4. Wordlist generation

Command	What it does
`crunch <min> <max> <charset> -t <patrón> -o <archivo>`	Creates a custom wordlist with min/max lengths, a charset of possible words and a pattern; saved to the file given with -o.
`python3 cupp.py -i`	Python script that builds a wordlist from target data (family names, pets, etc.).

Pattern characters (crunch)

- @: lowercase letters
- ,: uppercase letters
- %: numbers
- ^: special symbols

Example: "DOMAIN@@@@19%%" (the same example as above with this tool's characters).

5. Rules

Command	What it does
`hashcat -a 0 -m <modulo> hash.txt <diccionario> -r rule.txt`	Applies the rules in rule.txt to the dictionary words. The -r flag works for any attack type.
`hashcat -a 0 -m <modulo> hash.txt <diccionario> -g 1000`	Same as above, but with 1000 randomly auto-generated rules (more rules = higher hit rate, but longer time).

Main rules (with example)

- l: all to lowercase — Domain → domain
- u: all to uppercase — Domain → DOMAIN

- c: first uppercase, rest lowercase — domain → Domain
- C: first lowercase, rest uppercase — Domain → dOMAIN
- t: toggle case — DoMaIn → dOmAiN
- TN: toggle char N — T2: DoMaIn → DOMaIn
- d: duplicate word — Domain → DomainDomain
- q: duplicate each char — Domain → DDoommaaiinn
- zN: duplicate first char — Domain → DDomain
- ZN: duplicate last char — Domain → Domainn
- {: rotate word left — Domain → omainD
- }: rotate word right — Domain → nDomai
- ^N: prepend N — ^!: Domain → !Domain
- $N: append N — $!: Domain → Domain!
- r: reverse word — Domain → niamoD
- sNX: replace N with X — so@: Domain → D@main

Concrete example: `c so0 sil shH ss5 $2 $0 %2 $6` → capitalizes the first letter, replaces o→0, i→l, h→H, s→5 and appends 2026 at the end.